

LU QUICKSORT E' UN ESEMPIO DI ALGORITMO RANDOMIZZATO CAS UEGAS, OVEIRO UN ALGORITMO CHE FORNISCE SEMPRE L'OUTPUT CORRETTO ANCHE SE NON E' ESEGUITO NELLO STESSO TEMPO SULO STESSO INPUT. IL TEMPO DI ESECUZIONE NON DIPENDE DALL'INPUT MA DAL CAMPIONAMENTO ED E' QUINDI UNA VARIABILE CASUALE.

L'IDEA DI BASE E' SIMILE AL QUICKSORT DETERMINISTICO, LA DIFFERENZA CHE LA POSIZIONE DEL PIVOT NON E' SEMPRE LA STESSA. IL CASO PEGGIORE DI LU QUICKSORT E' $O(n^2)$.

STIMIAMO ALLORA IL VALORE ATTESO DEL TEMPO D. ESECUZIONE DI LU QUICKSORT (INTERO COME NUMERO ATTESO DEI CONFRONTI ESPERIMENTALI. PER SEMPLICITA' ASSUMIAMO CHE S SIA COSTITUITO DAI NUMERI NATURALI DA 1 A m . INTRODUCIAMO LA VARIABILE INDICATRICE $I(i, j)$. IL SUO OBBIETTIVO E' "INSERIRE" SE UN DETERMINATO EVENTO SI E' VERIFICATO OPRMA NO. VALE 1 SE L'EVENETO CHE STIAMO OSSERVANDO ACCADE. VALE 0 SE L'EVENETO NON ACCADE. QUINDI:

$$I(i, j) = \begin{cases} 1 & \text{SE } i \text{ E } j \text{ SONANO CONFRONTATI} \\ 0 & \text{ALTRIMENTI} \end{cases}$$

QUESTO ACCADE QUANDO IL PIVOT E' i O j .

IL NUMERO DI CONFRONTI ESPERIMENTALI E' DATO ALLORA DALLA VARIABILE CASUALE:

$$X = \sum_{i=1}^{m-1} \sum_{j=i+1}^m I(i, j) \quad *$$

QUESTA FORMULA DEFINISCE LA VARIABILE CASUALE X , CHE E' IL NUMERO TOTALE DI CONFRONTI ESEGUITI IN UNA SINGOLA ESECUZIONE DELL'ALGORITMO.

Il ruolo delle sommatorie: Generare le coppie

Le due sommatorie $\sum_{i=1}^{n-1} \sum_{j=i+1}^n$ sono un modo formale per dire: "Prendi in considerazione tutte le possibili coppie di elementi (i, j) tali che i sia più piccolo di j ."

Esempio con $n = 4$ (elementi $\{1, 2, 3, 4\}$): Le sommatorie ti dicono di "visitare" queste coppie:

- $i = 1$: j va da 2 a 4 $\Rightarrow (1, 2), (1, 3), (1, 4)$
- $i = 2$: j va da 3 a 4 $\Rightarrow (2, 3), (2, 4)$
- $i = 3$: j va da 4 a 4 $\Rightarrow (3, 4)$

Le coppie totali sono $(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)$.

Cosa stai sommando: Il valore di $I(i, j)$

$I(i, j)$ è il termine che viene sommato. Non è un fattore di moltiplicazione.

È una variabile che può valere solo 1 o 0.

Riprendendo l'esempio con $n = 4$, la formula $X = \sum_{i=1}^3 \sum_{j=i+1}^4 I(i, j)$ è solo una forma contratta per scrivere:

$$X = I(1, 2) + I(1, 3) + I(1, 4) + I(2, 3) + I(2, 4) + I(3, 4)$$

Non c'è nessuna moltiplicazione! Stai solo sommando i valori di questi 6 "interruttori".

Facciamo un esempio di esecuzione: Immagina che il tuo algoritmo Quicksort (per $n = 4$) faccia questi confronti:

- Confronta 1 e 3? Si $\Rightarrow I(1, 3) = 1$
- Confronta 1 e 2? No $\Rightarrow I(1, 2) = 0$
- Confronta 1 e 4? Si $\Rightarrow I(1, 4) = 1$
- Confronta 2 e 3? No $\Rightarrow I(2, 3) = 0$
- Confronta 2 e 4? No $\Rightarrow I(2, 4) = 0$
- Confronta 3 e 4? Si $\Rightarrow I(3, 4) = 1$

Ora calcoli X sommando questi 1 e 0: $X = 0 + 1 + 1 + 0 + 0 + 1 = 3$

In questa esecuzione, il numero totale di confronti è stato 3.

E' IMPORTANTE PER MISURARE IL

TEMPO DI ESECUZIONE

ANALISI CASO PEGGIORE

SI RICORDA NEL CASO PEGGIORE SE IL PIVOT È SEMPRE IL MINIMO (O MASSIMO) DI CONSEGUENZA L'ARRAY NON SI DIVIDE MAI. OGNI ELEMENTO FINISCE PER ESSERE CONFRONTATO CON OGNI ALTRO ELEMENTO. SULLA LINEA $I(i, j) = 1$ LA FORMULA DIVENTA "SOMMA 1 PER OGNI POSSIBILE COPPIA (i, j) CON $i < j$ ". QUANTE COPPIE DISTINTE POSSO FORMARE CON m ELEMENTI?

$$\frac{m(m-1)}{2}$$

PER LA PROPRIETÀ DELLA LINEARITÀ DEL VALORE ATTESO DELL'UGUAGLIANZA (*) È TENUTO CONTO DEL FATTO CHE LE $I(i, j)$ SONO VARIABILI INDIPENDENTI, SE $p_{ij}(1)$ È LA PROBABILITÀ CHE i E j SARANNO CONFRONTATI, IL VALORE ATTESO DEL NUMERO DI CONFRONTI X È:

$$E[X] = E\left[\sum_{i=1}^{m-1} \sum_{j=i+1}^m I(i, j)\right] = \sum_{i=1}^{m-1} \sum_{j=i+1}^m p_{ij}(1)$$

VALUTIAMO $p_{ij}(1)$. INIZIALMENTE TUTTI I $j-i+1$ NUMERI DA i A j SONO NELLA STESSA PARTIZIONE.

IL DESTINO DELLA COPPIA (i, j) (SE SARANNO CONFRONTATI O NO) SI DECIDE LA PRIMA VOLTA CHE L'ALGORITMO SCEGLIE UN PIVOT ALL'INTERNO DI S . SE IL PIVOT È UN ELEMENTO INTERMEDIO $i < k < j$ QUELLO k DIVIDE i E j IN DUE PARTIZIONI, ANESSA E NON SARANNO MAI CONFRONTATI. NEL CASO INVECE CHE $k=i$ O $k=j$ ALLORA i E j SONO CONFRONTATI. QUINDI:

• CASI TOTALI POSSIBILI: $j-i+1$

• CASI FAVOREVOLI: 2 (solo i o j)

ASSUMENDO CHE LA SCELTA DEL PIVOT SIA CASUALE (EQUIPROBABILITÀ) ALLORA LA PROBABILITÀ È:

$$p_{ij}(1) = \frac{\text{CASI FAVOREVOLI}}{\text{CASI TOTALI}} = \frac{2}{j-i+1}$$

Ora che abbiamo p_{ij} , dobbiamo calcolare $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n p_{ij}$. Il testo inizia risolvendo la somma interna (quella su j).

$$\sum_{j=i+1}^n \frac{2}{j-i+1}$$

Questa è una somma complicata (è $2 \times (1/2 + 1/3 + 1/4 + \dots)$). Per analizzarla per n grande, il testo fa l'approssimazione con un integrale, che è un metodo standard.

- La Somma: $\sum_{j=i+1}^n \frac{2}{j-i+1}$
- L'Approssimazione: $\approx \int_{i+1}^n \frac{2}{t-i+1} dt$
- Il Risultato: Risolvendo l'integrale si ottiene $2 \ln(n-i+1) - 2 \ln(2)$. Per n grande, si approssima ulteriormente a $\approx 2 \ln(n-i+1)$ (ignorando la costante $2 \ln(2)$).

Quindi, il "contributo" totale di confronti per un singolo elemento i è circa $2 \ln(n-i+1)$.

3. *** Calcolo Finale di $E[X]$ (La Somma Esterna)

Ora sostituiamo il risultato della somma interna nella formula principale:

$$E[X] = \sum_{i=1}^{n-1} \left(\sum_{j=i+1}^n p_{ij} \right) \approx \sum_{i=1}^{n-1} 2 \ln(n-i+1)$$

Di nuovo, abbiamo una somma complicata. E di nuovo, la approssimiamo con un integrale:

- La Somma: $\sum_{i=1}^{n-1} 2 \ln(n-i+1)$
- L'Approssimazione: $\approx \int_1^{n-1} 2 \ln(n-t+1) dt$
- La Sostituzione: Come dice il testo, si usa la sostituzione $y = n-t+1$ per risolvere l'integrale. Questo trasforma l'integrale in $\int_2^n 2 \ln(y) dy$.
- La Soluzione: L'integrale di $\ln(y)$ è $y \ln y - y$. Valutando l'espressione $2[y \ln y - y]$ tra n e 2 si ottiene: $2(n \ln n - n) - 2(2 \ln 2 - 2)$
- Il Risultato Finale: L'espressione è $2n \ln n - 2n - 4 \ln 2 + 4$. Quando n diventa molto grande, l'unico termine che conta davvero (il "termine dominante") è $2n \ln n$.

Il testo quindi conclude: $E[X] \approx 2n \ln n$

Visto che $\ln n$ (logaritmo naturale) e $\log_2 n$ (logaritmo binario) differiscono solo per una costante, in notazione O-grande si scrive:

$$E[X] = O(n \log n)$$

Questo dimostra matematicamente che il numero medio di confronti di LQuickSort è $O(n \log n)$, che è efficiente quanto MergeSort!